

COMPUTER SYSTEM ARCHITECTURE

LAB PRACTICAL



FACULTY OF TECHNOLOGY

UNIVERSITY OF DELHI

NEW DELHI - 110007

SUBMITTED TO:

MS. BHAWNA RAWAT

SUBMITTED BY:

AMITANSH KESHARWANI

24293916055 (CSE-A)

Experiment - 1

Aim: Installation, familiarization with CPUSim software and creating a 16-bit base machine.

Software: CPUSim 3.6.9 (or latest version) — A Java-based computer simulation tool used to design and simulate simple computer architectures at the register and microinstruction level.

Theory:

CPUSim is an educational simulation tool that allows users to design and simulate simple computer architectures. It provides a graphical interface to define registers, memory, microinstructions, and machine instructions.

A **16-bit base machine** refers to a computer architecture where the data bus, registers, and memory addresses are 16 bits wide. This means the machine can process 16 bits of data at a time and can address up to $2^{16} = 65,536$ memory locations.

Key components defined in a 16-bit base machine in CPUSim:

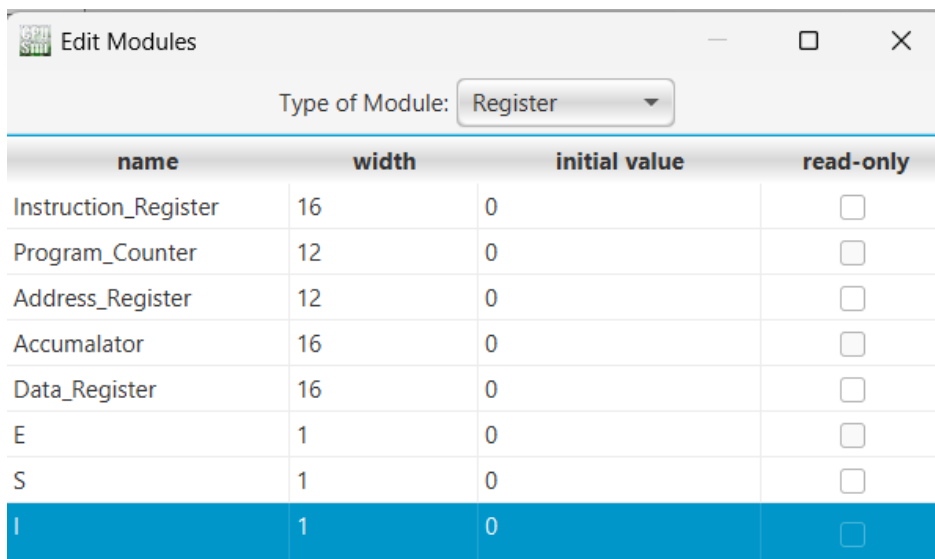
- **Registers:** Storage units inside the CPU. For a 16-bit machine, registers like PC (Program Counter), ACC (Accumulator), IR (Instruction Register), and MAR/MDR are each 16 bits wide.
- **ALU (Arithmetic Logic Unit):** Performs arithmetic and logical operations on 16-bit data.
- **Memory:** RAM is configured with a word size of 16 bits.
- **Microinstructions:** Low-level control signals that define how data moves between components during each clock cycle.
- **Machine Instructions:** High-level instructions (like LOAD, STORE, ADD) that are broken down into sequences of microinstructions.

Steps to create a 16-bit base machine in CPUSim:

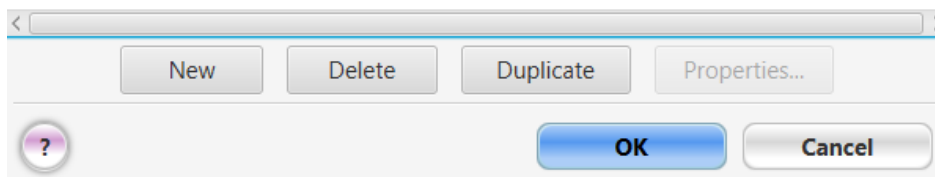
1. Launch CPUSim and open a new machine.

2. Define registers with a width of 16 bits (PC, ACC, IR, MAR, MDR, etc.).
3. Configure RAM with 16-bit word size and appropriate address length.
4. Define condition bits and the ALU operations.
5. Create microinstructions to control data flow.
6. Define machine instructions mapped to their microinstruction sequences.
7. Save and test the machine by running a simple program.

Implementation Screenshots:



name	width	initial value	read-only
Instruction_Register	16	0	☐
Program_Counter	12	0	☐
Address_Register	12	0	☐
Accumulator	16	0	☐
Data_Register	16	0	☐
E	1	0	☐
S	1	0	☐
I	1	0	☐



 Edit Modules

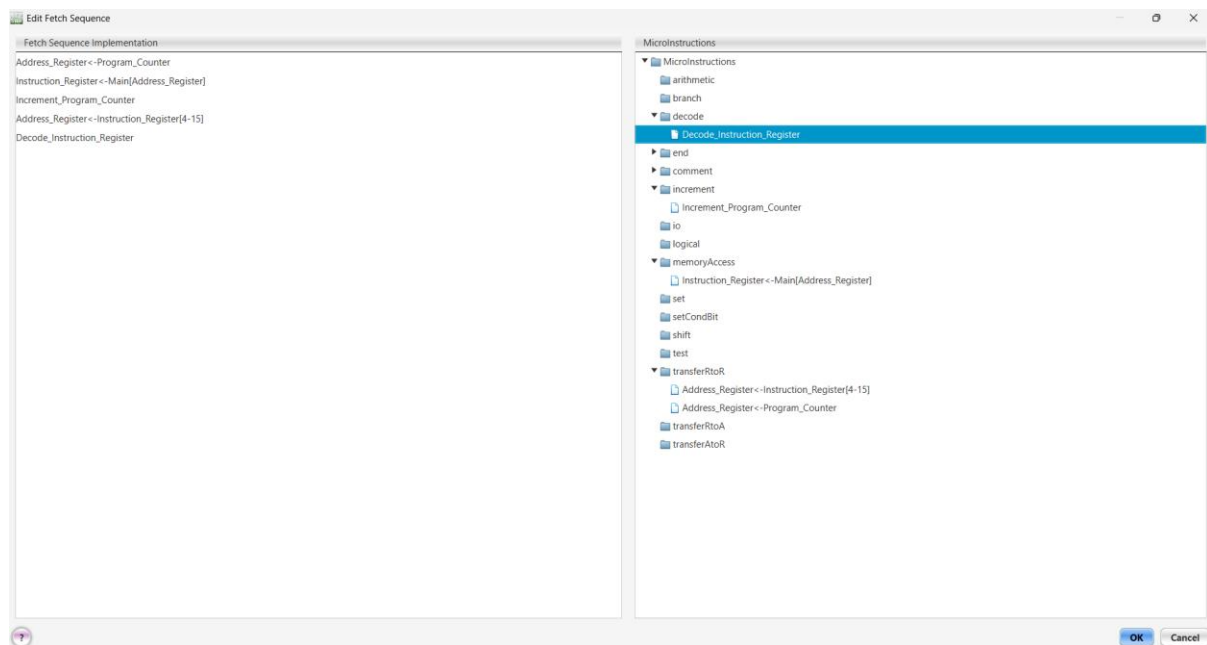
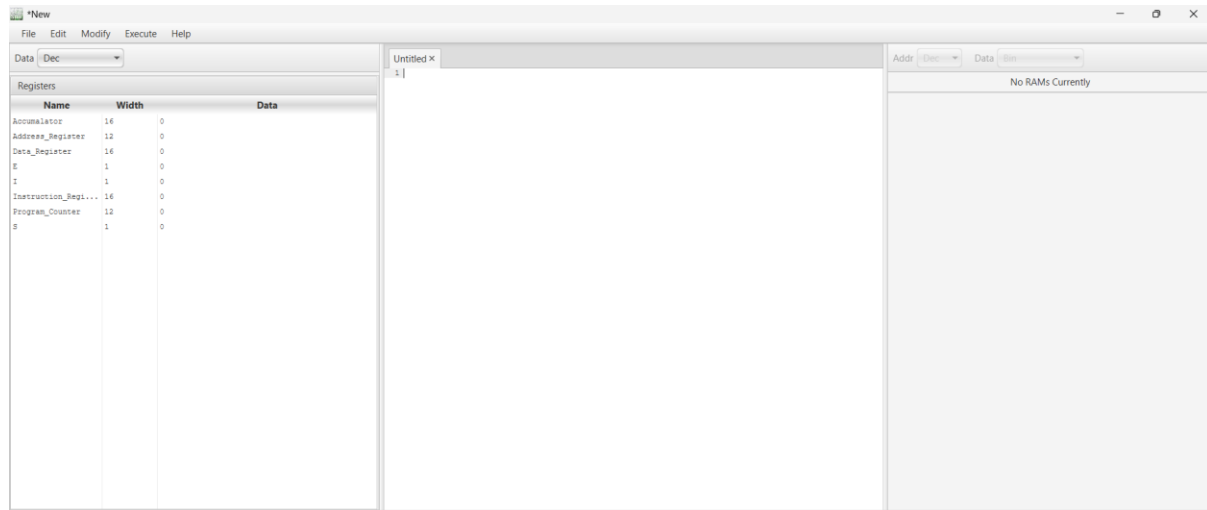
Type of Module: ConditionBit ▾

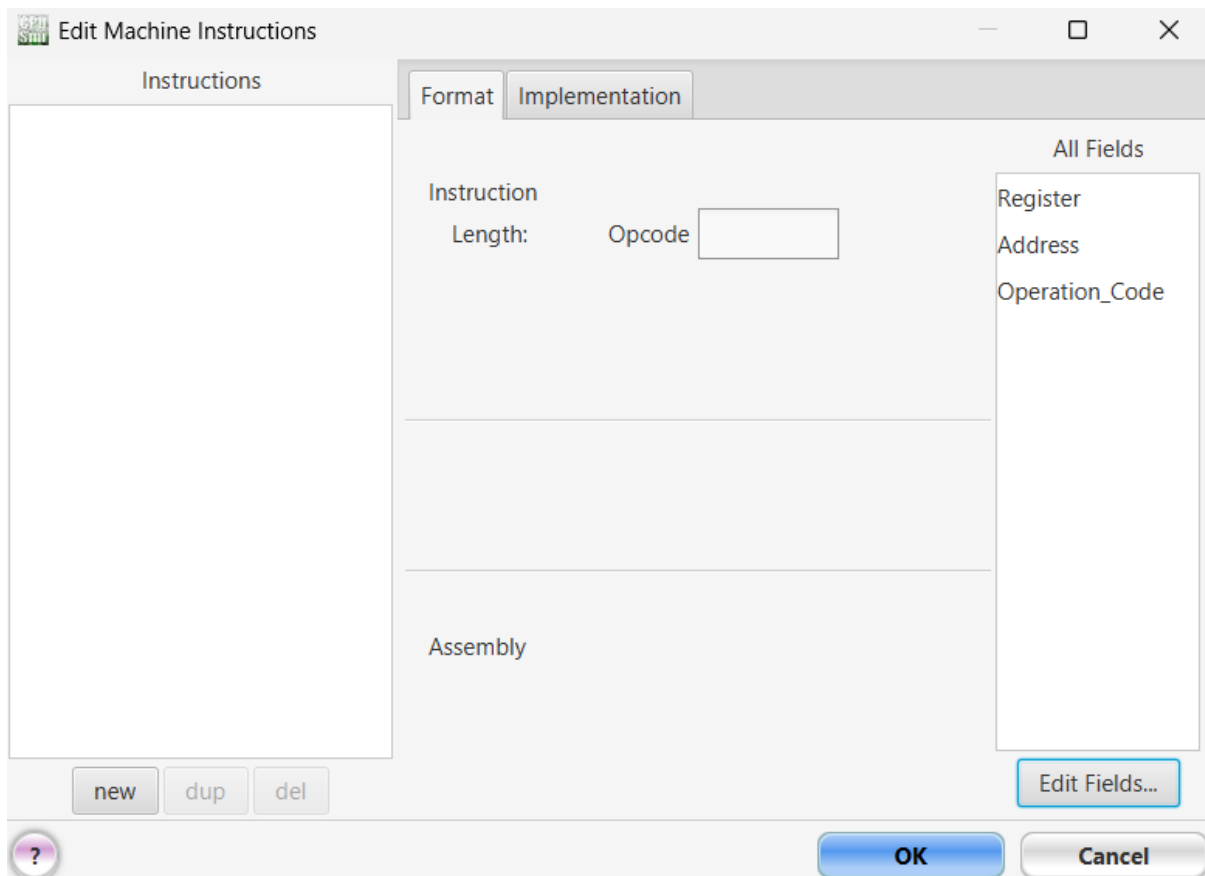
name	register	bit	halt
Carry_Bit	E	0	☐
Halt_Bit	S	0	☒

< >

New Delete Duplicate Properties...

 OK Cancel





Results:

A 16-bit base machine was successfully created in CPUSim. Registers of 16-bit width were defined, RAM was configured accordingly, and machine instructions were mapped to their corresponding microinstruction sequences. The machine was verified to be functional by executing a basic instruction set.

Learning Outcomes:

After completing this experiment, students are able to:

1. Understand the installation and basic interface of the CPUSim software.
2. Define and configure registers, memory, and ALU for a custom CPU architecture.
3. Create microinstructions and map them to machine-level instructions.

Experiment - 2

Aim: Implementing ADD and SUBTRACT operations on the 16-bit base machine in CPUSim.

Software: CPUSim 3.6.9 (or latest version)

Theory:

Arithmetic operations like **Addition** and **Subtraction** are fundamental operations performed by the ALU (Arithmetic Logic Unit) of any CPU. In CPUSim, these operations are implemented by defining appropriate machine instructions and their corresponding microinstruction sequences.

Addition (ADD): Addition is the process of summing two operands. In a 16-bit machine, two 16-bit values stored in registers or memory are fetched and added together using the ALU. The result is stored back in the Accumulator (ACC) register.

- Operation: $ACC \leftarrow ACC + \text{Operand}$
- If the result exceeds 16 bits, an overflow condition is raised.

Subtraction (SUB): Subtraction finds the difference between two operands. In most CPU architectures, subtraction is implemented using **2's complement addition**, i.e., subtracting B from A is equivalent to adding A and the 2's complement of B.

- Operation: $ACC \leftarrow ACC - \text{Operand}$
- If the result is negative, the sign bit (MSB) is set to 1 in 2's complement representation.

Fetch-Decode-Execute Cycle: Every machine instruction in CPUSim follows the fetch-decode-execute cycle:

1. **Fetch:** The instruction is fetched from memory using the Program Counter (PC).
2. **Decode:** The instruction is decoded to identify the operation and operands.

3. **Execute:** The ALU performs the required ADD or SUB operation and stores the result.

Microinstruction Sequence for ADD:

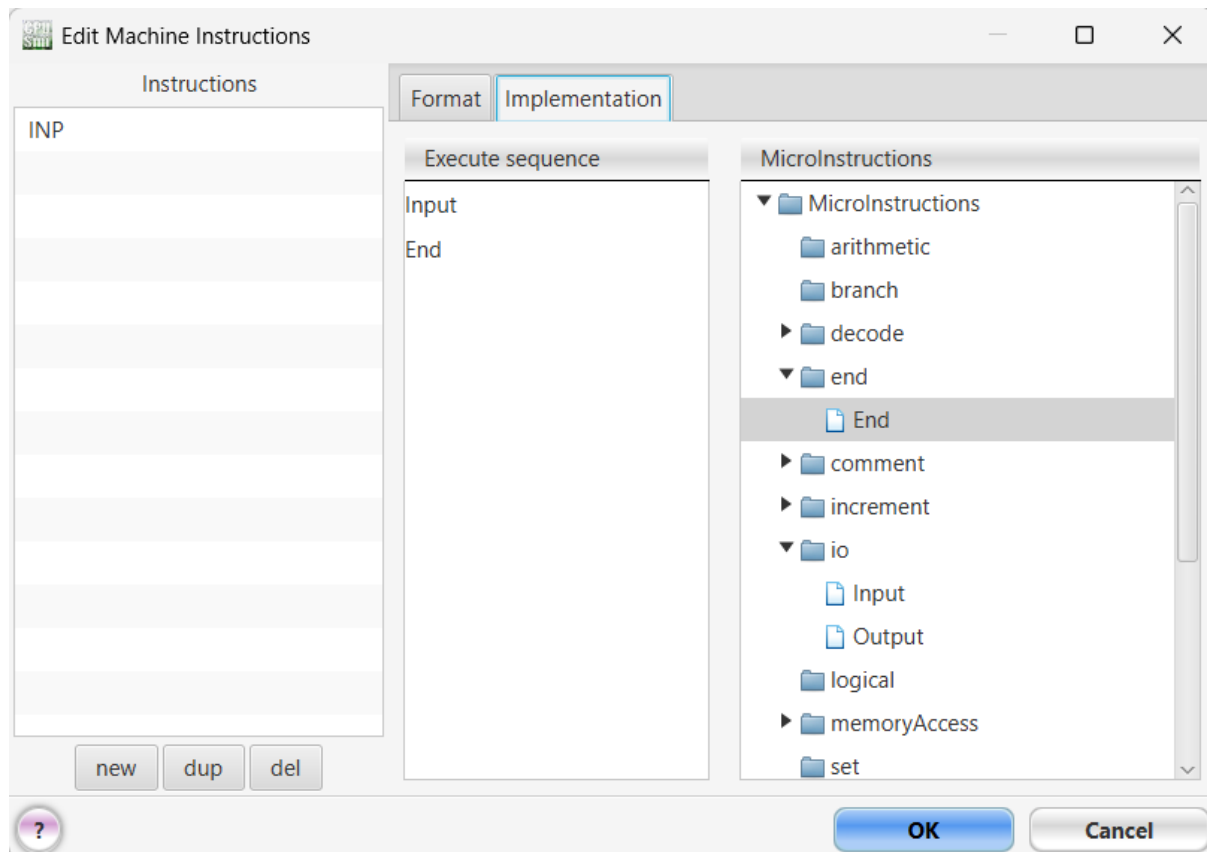
1. $MAR \leftarrow PC$ (fetch instruction address)
2. $MDR \leftarrow \text{Memory}[MAR]$, $PC \leftarrow PC + 1$
3. $IR \leftarrow MDR$ (load instruction into IR)
4. $MAR \leftarrow \text{Address field of IR}$
5. $MDR \leftarrow \text{Memory}[MAR]$ (fetch operand)
6. $ACC \leftarrow ACC + MDR$ (perform addition)

Microinstruction Sequence for SUB:

1. $MAR \leftarrow PC$
 2. $MDR \leftarrow \text{Memory}[MAR]$, $PC \leftarrow PC + 1$
 3. $IR \leftarrow MDR$
 4. $MAR \leftarrow \text{Address field of IR}$
 5. $MDR \leftarrow \text{Memory}[MAR]$ (fetch operand)
 6. $ACC \leftarrow ACC - MDR$ (perform subtraction)
-

Implementation Screenshots:

ADD Operation-



add_operation

File Edit Modify Execute Help

Data Hex

Registers

Name	Width	Data
Accumulator	16	0000
Address_Re...	12	000
Data_Register	16	0000
E	1	0
I	1	0
Instructio...	16	0000
Program_Co...	12	000
S	1	0

add.a ×

```

1 START: INP
2 STA NUM
3 INP
4 ADD NUM
5 OUT
6 HLT
7 NUM: .data 1 d

```

Addr Hex Data Hex

Main

Addr	Data
000	0000
001	0000
002	0000
003	0000
004	0000
005	0000
006	0000
007	0000
008	0000
009	0000
00A	0000
00B	0000
00C	0000
00D	0000
00E	0000
00F	0000
010	0000
011	0000
012	0000
013	0000
014	0000
015	0000
016	0000
017	0000

Edit Microinstructions

Type of Microinstruction:

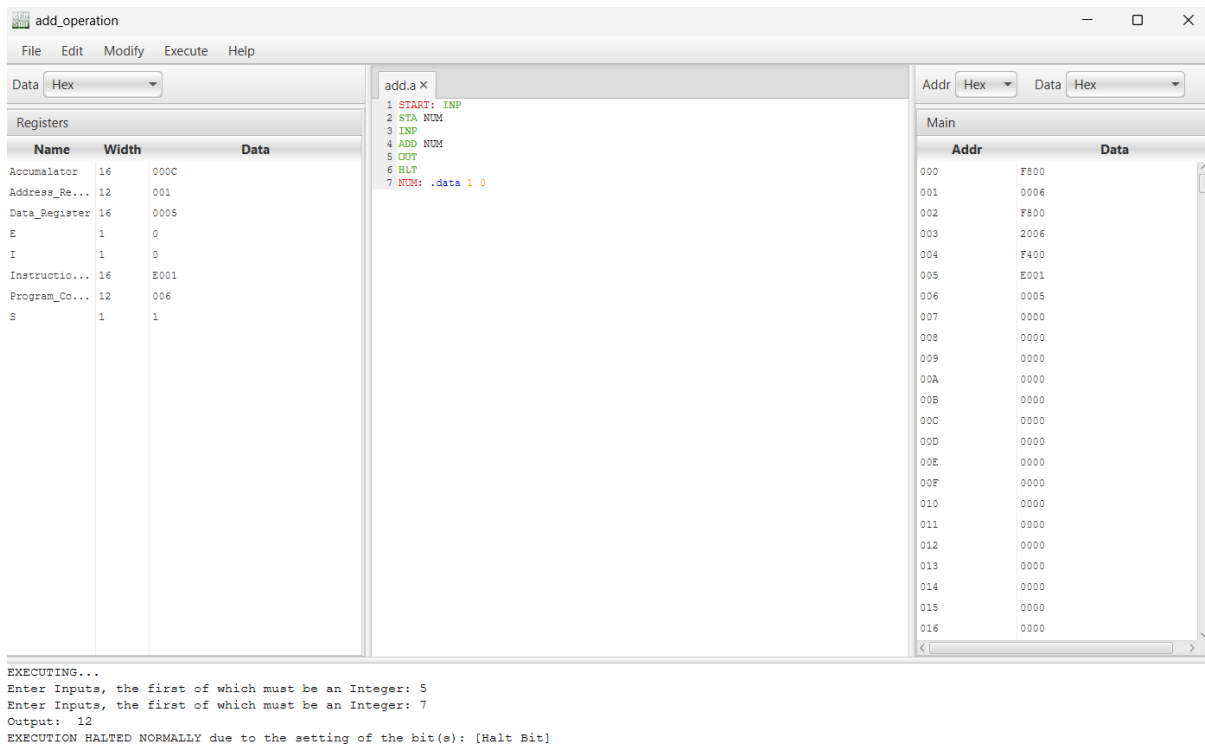
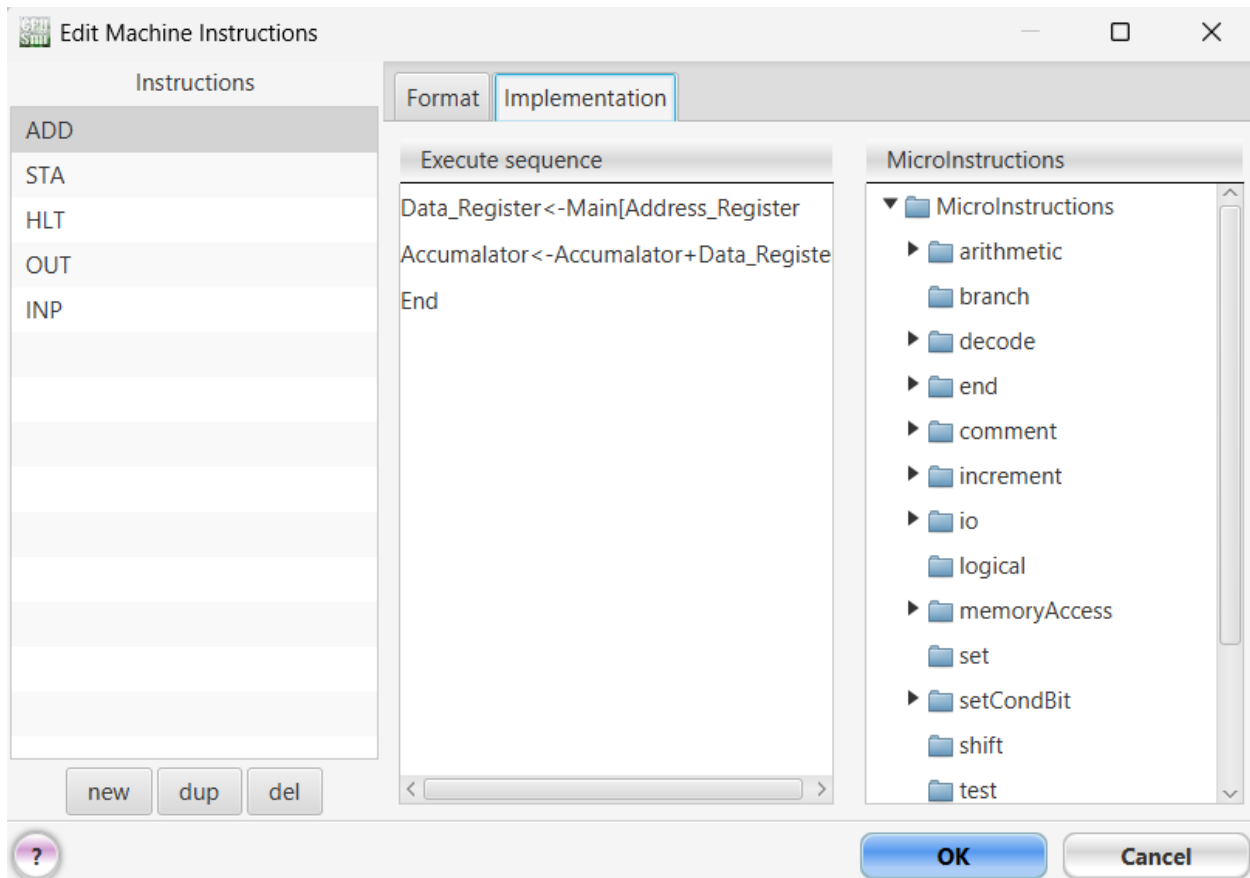
MemoryAccess

name	direction	memory	data	address
Main[Address_Re...	write	Main	Accumalator	Address_Register
Instruction_Regist...	read	Main	Instruction_Register	Address_Register

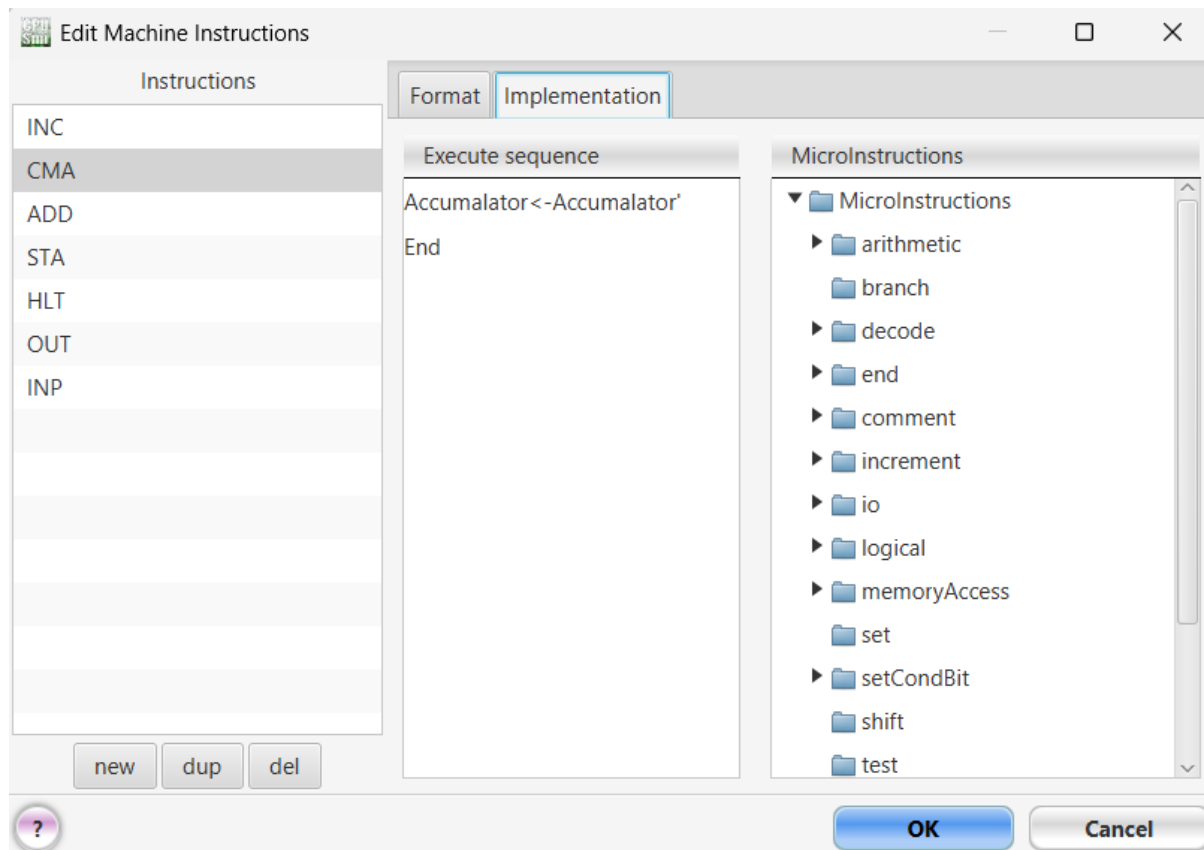
NewDeleteDuplicate

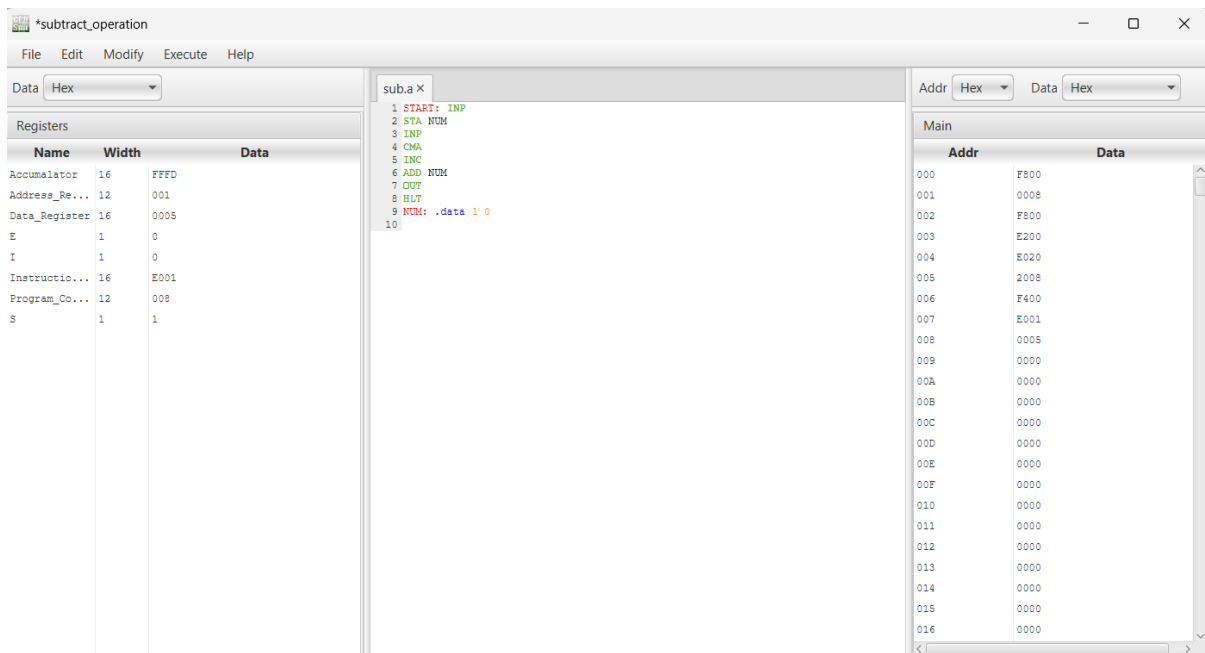
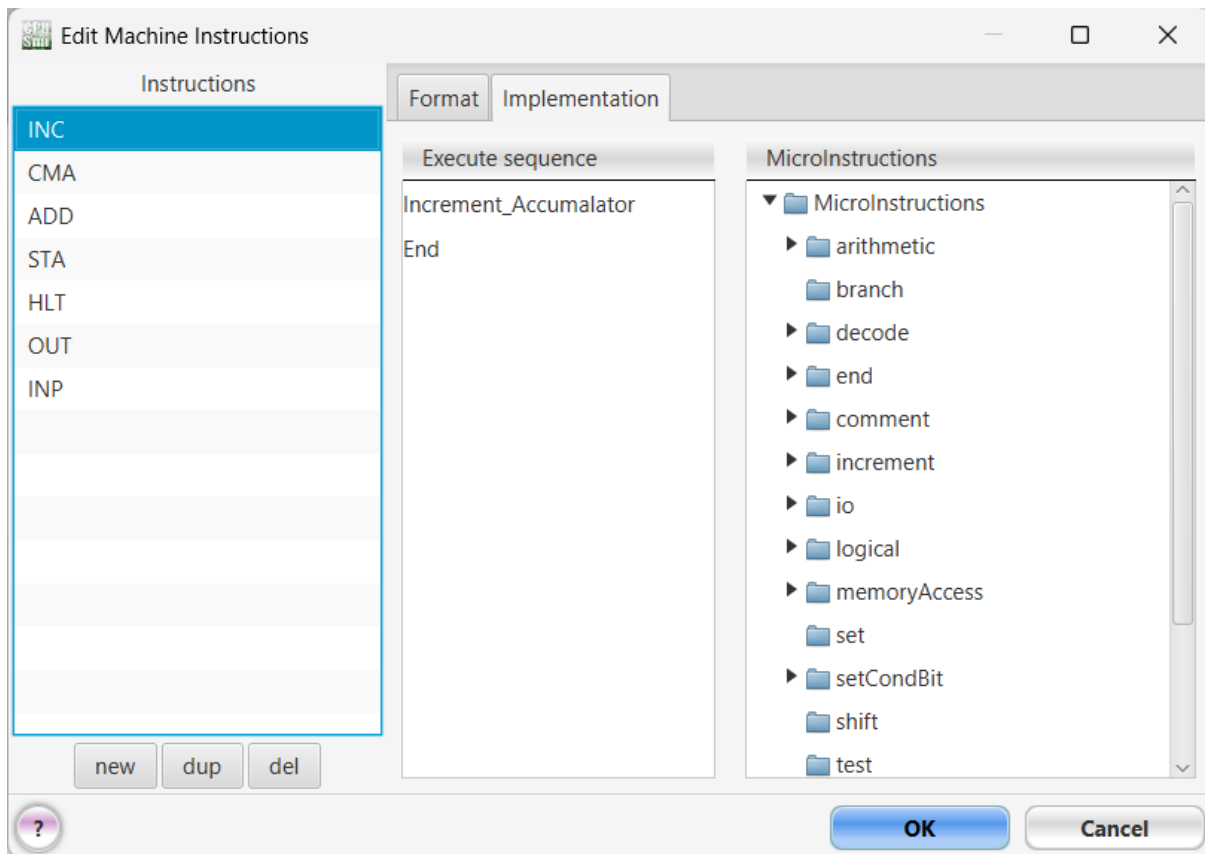
?

OKCancel



SUBTRACT Operation-





EXECUTING...

Enter Inputs, the first of which must be an Integer: 5

Enter Inputs, the first of which must be an Integer: 8

Output: -3

EXECUTION HALTED NORMALLY due to the setting of the bit(s): [Halt_Bit]

Results:

The ADD and SUBTRACT operations were successfully implemented on the 16-bit base machine in CPUSim. The ALU correctly computed the sum and difference of two 16-bit values. The results were stored in the Accumulator register and verified against expected outputs.

Learning Outcomes:

After completing this experiment, students are able to:

1. Understand how arithmetic operations are implemented at the microinstruction level in a CPU.
2. Define ADD and SUB machine instructions in CPUSim with correct microinstruction sequences.
3. Comprehend the role of the ALU, Accumulator, and memory in executing arithmetic operations.
4. Understand the fetch-decode-execute cycle and its application in instruction execution.
5. Verify the correctness of arithmetic operations by observing register states during simulation.

Experiment - 3

Aim: Implementing logical operations AND, OR, NOT, XOR, NOR, and NAND on the 16-bit base machine in CPUSim.

Software: CPUSim 3.6.9 (or latest version)

Theory:

Logical operations are bitwise operations performed by the ALU on binary data. They operate on each corresponding bit of the operands independently. In CPUSim, these operations are implemented by defining machine instructions and their microinstruction sequences for each logical operation.

1. AND Operation: The AND operation outputs 1 only when both corresponding bits of the operands are 1, otherwise outputs 0.

- Operation: $ACC \leftarrow ACC \text{ AND } \text{Operand}$
- Truth Table:

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

2. OR Operation: The OR operation outputs 1 when at least one of the corresponding bits is 1.

- Operation: $ACC \leftarrow ACC \text{ OR } \text{Operand}$

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

3. NOT Operation: The NOT operation is a unary operation that inverts each bit of the operand (1 becomes 0 and 0 becomes 1).

- Operation: $ACC \leftarrow NOT\ ACC$

A	NOT A
0	1
1	0

4. XOR Operation: The XOR (Exclusive OR) operation outputs 1 only when the corresponding bits of the two operands are different.

- Operation: $ACC \leftarrow ACC\ XOR\ Operand$

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

5. NOR Operation: The NOR operation is the complement of the OR operation. It outputs 1 only when both bits are 0.

- Operation: $ACC \leftarrow NOT\ (ACC\ OR\ Operand)$

A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

6. NAND Operation: The NAND operation is the complement of the AND operation. It outputs 0 only when both bits are 1.

- Operation: $ACC \leftarrow \text{NOT} (ACC \text{ AND Operand})$

A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

Microinstruction Sequence (General for binary logical operations):

1. $MAR \leftarrow PC$
2. $MDR \leftarrow \text{Memory}[MAR], PC \leftarrow PC + 1$
3. $IR \leftarrow MDR$ (decode instruction)
4. $MAR \leftarrow \text{Address field of } IR$
5. $MDR \leftarrow \text{Memory}[MAR]$ (fetch operand)
6. $ACC \leftarrow ACC \text{ (operation) } MDR$ (perform logical operation)

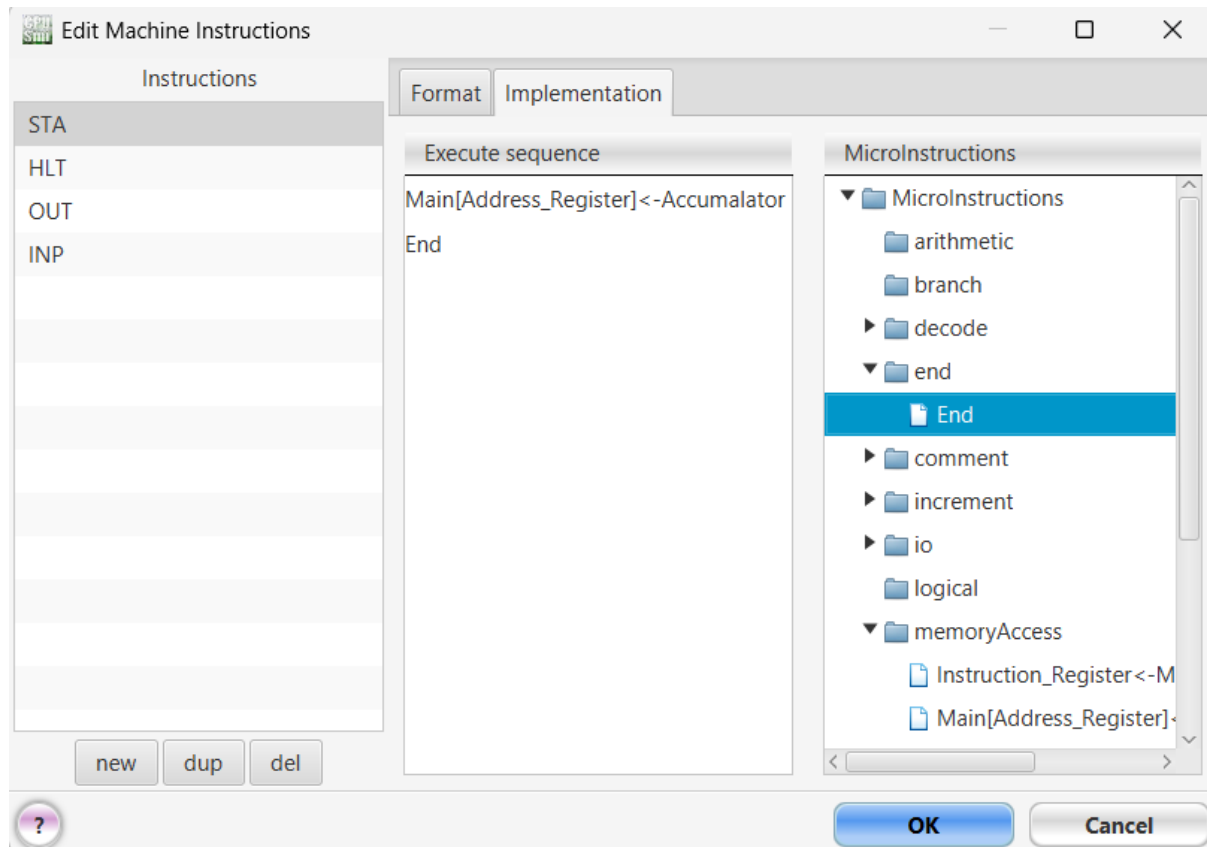
For NOT (unary operation):

1. $MAR \leftarrow PC$
2. $MDR \leftarrow \text{Memory}[MAR], PC \leftarrow PC + 1$

3. $IR \leftarrow MDR$
4. $ACC \leftarrow \text{NOT } ACC$ (invert accumulator bits directly)

Implementation Screenshots:

AND Operation-



Edit Microinstructions

Type of Microinstruction:

Logical

name	type	source1	source2	destination
Accumalator<-Ac...	AND	Accumalator	Data_Register	Accumalator

New

Delete

Duplicate

?

OK

Cancel

Edit Microinstructions

Type of Microinstruction:

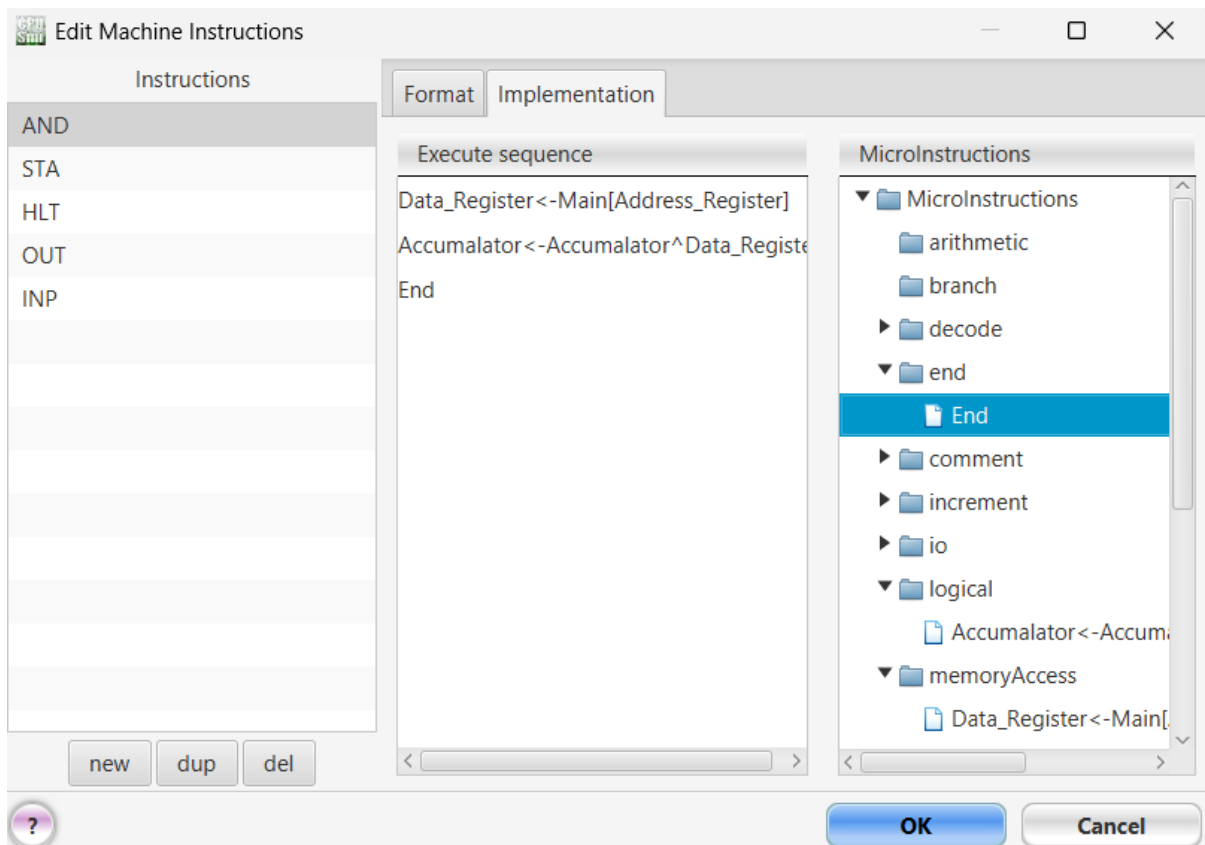
MemoryAccess

name	direction	memory	data	address
Data_Register<-...	read	Main	Data_Register	Address_Register
Instruction_Regist...	read	Main	Instruction_Register	Address_Register
Main[Address_Re...	write	Main	Accumalator	Address_Register

NewDeleteDuplicate

?

OKCancel



```
and_operation.a ×
1 INP
2 STA NUM
3 INP
4 AND NUM
5 OUT
6 HLT
7
8 NUM: .data 1 0
9
```

```
EXECUTING...
Enter Inputs, the first of which must be an Integer: 0
Enter Inputs, the first of which must be an Integer: 0
Output: 0
EXECUTION HALTED NORMALLY due to the setting of the bit(s): [Halt_Bit]
```

```
EXECUTING...
Enter Inputs, the first of which must be an Integer: 1
Enter Inputs, the first of which must be an Integer: 1
Output: 1
EXECUTION HALTED NORMALLY due to the setting of the bit(s): [Halt_Bit]
```

EXECUTING...

Enter Inputs, the first of which must be an Integer: 1

Enter Inputs, the first of which must be an Integer: 0

Output: 0

EXECUTION HALTED NORMALLY due to the setting of the bit(s): [Halt_Bit]

OR Operation-

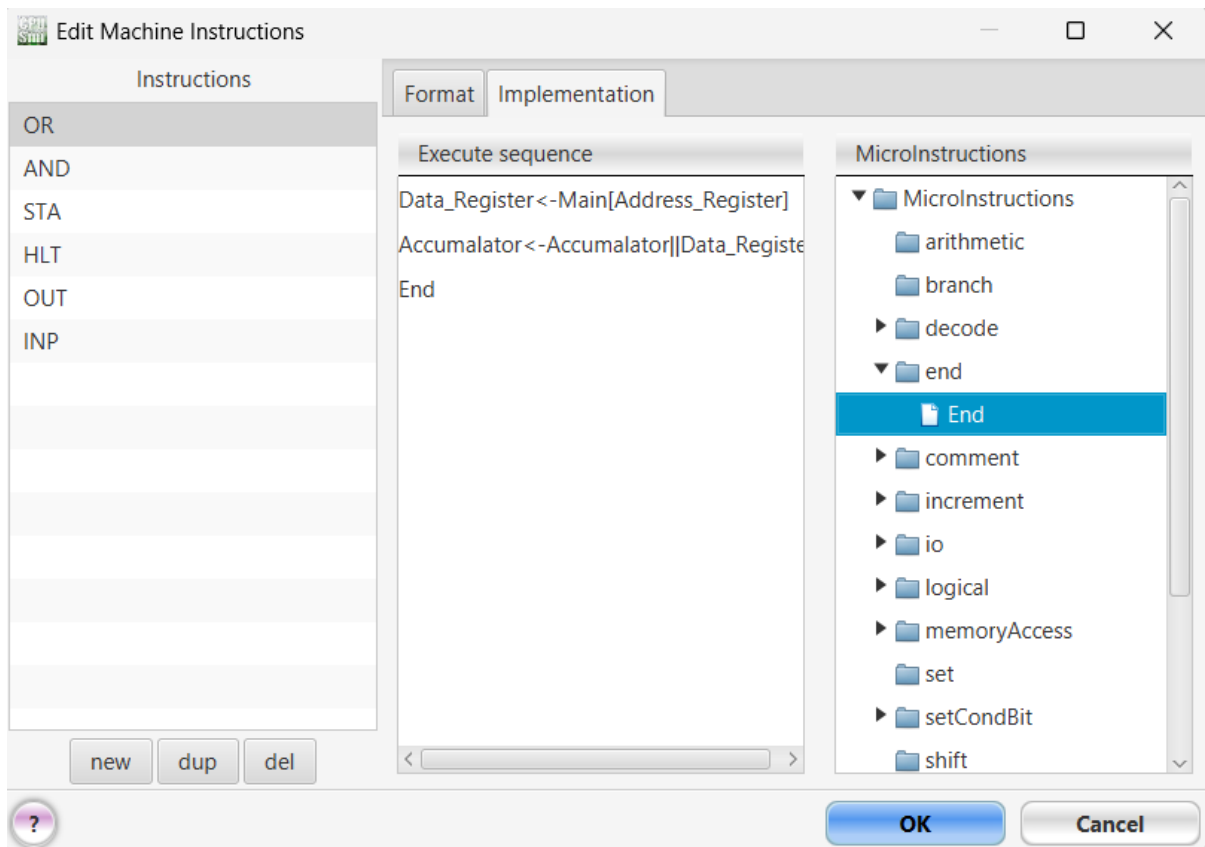
Edit Microinstructions

Type of Microinstruction: Logical

name	type	source1	source2	destination
Accumalator<-Ac...	OR	Accumalator	Data_Register	Accumalator
Accumalator<-Ac...	AND	Accumalator	Data_Register	Accumalator

New Delete Duplicate

? OK Cancel



```

*or.a x
1 INP
2 STA NUM
3 INP
4 OR NUM
5 OUT
6 HLT
7
8 NUM: .data 1 0
9

```

```

EXECUTING...
Enter Inputs, the first of which must be an Integer: 0
Enter Inputs, the first of which must be an Integer: 1
Output: 1
EXECUTION HALTED NORMALLY due to the setting of the bit(s): [Halt_Bit]

```

```

EXECUTING...
Enter Inputs, the first of which must be an Integer: 1
Enter Inputs, the first of which must be an Integer: 1
Output: 1
EXECUTION HALTED NORMALLY due to the setting of the bit(s): [Halt_Bit]

```

```

EXECUTING...
Enter Inputs, the first of which must be an Integer: 0
Enter Inputs, the first of which must be an Integer: 0
Output: 0
EXECUTION HALTED NORMALLY due to the setting of the bit(s): [Halt_Bit]

```

NOT Operation-

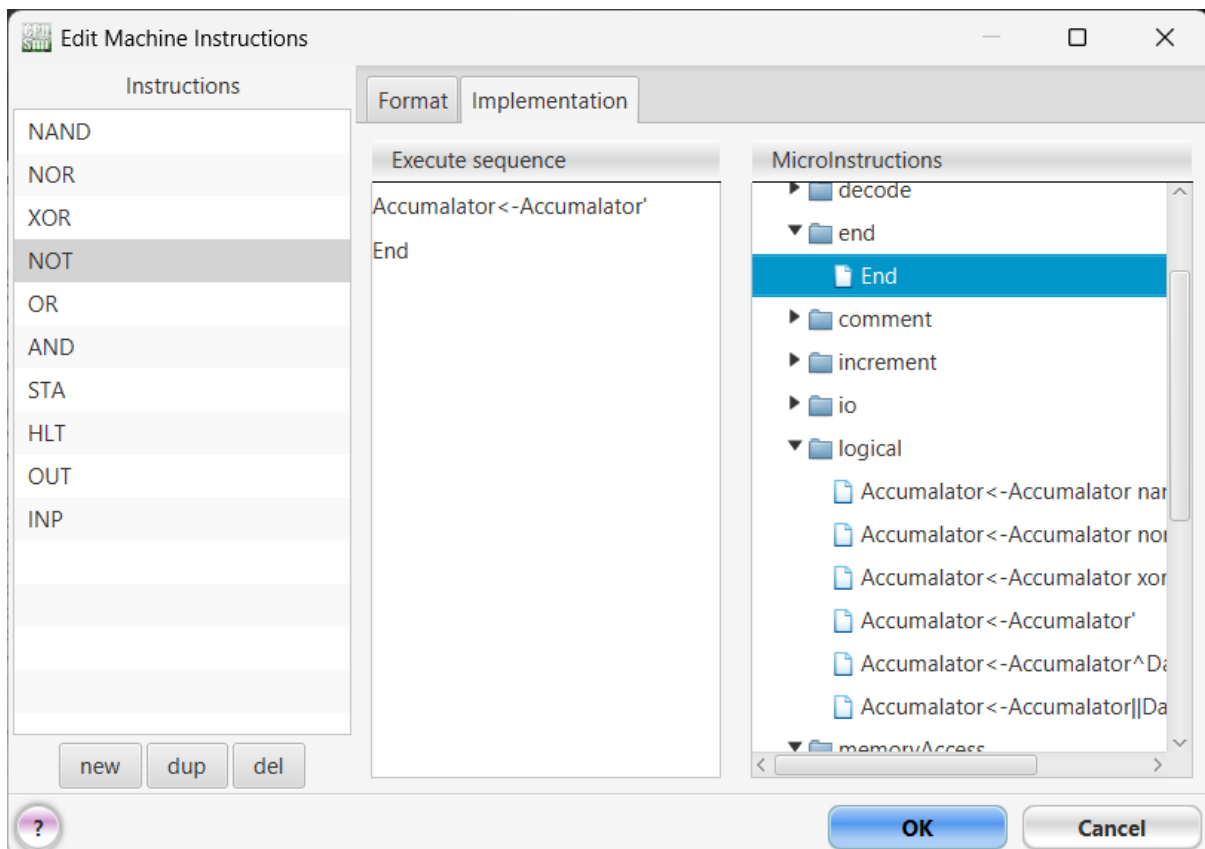
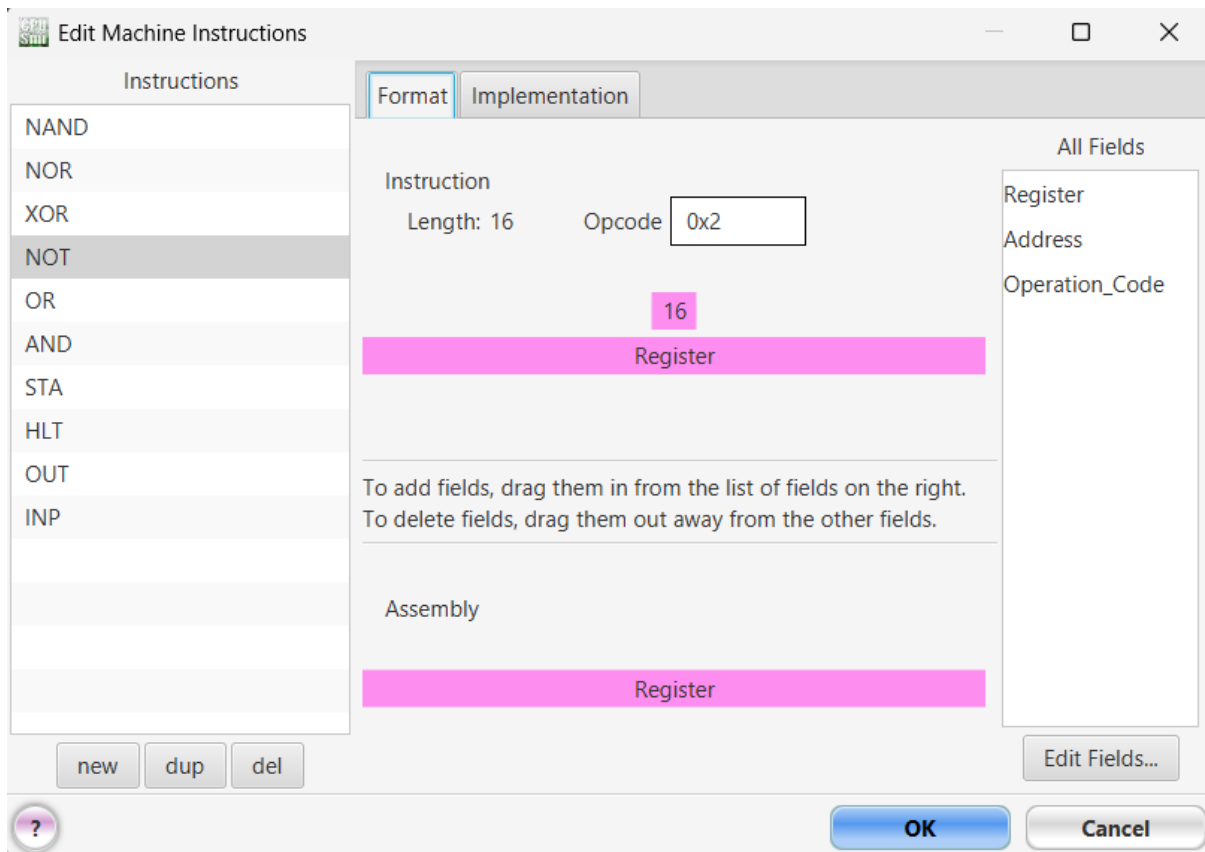
Edit Microinstructions

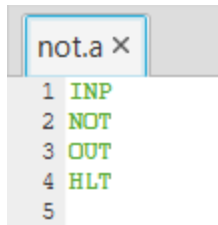
Type of Microinstruction: Logical

name	type	source1	source2	destination
Accumalator<-Ac...	NOT	Accumalator	Accumalator	Accumalator
Accumalator<-Ac...	AND	Accumalator	Data_Register	Accumalator
Accumalator<-Ac...	OR	Accumalator	Data_Register	Accumalator

New
Delete
Duplicate

?
OK
Cancel

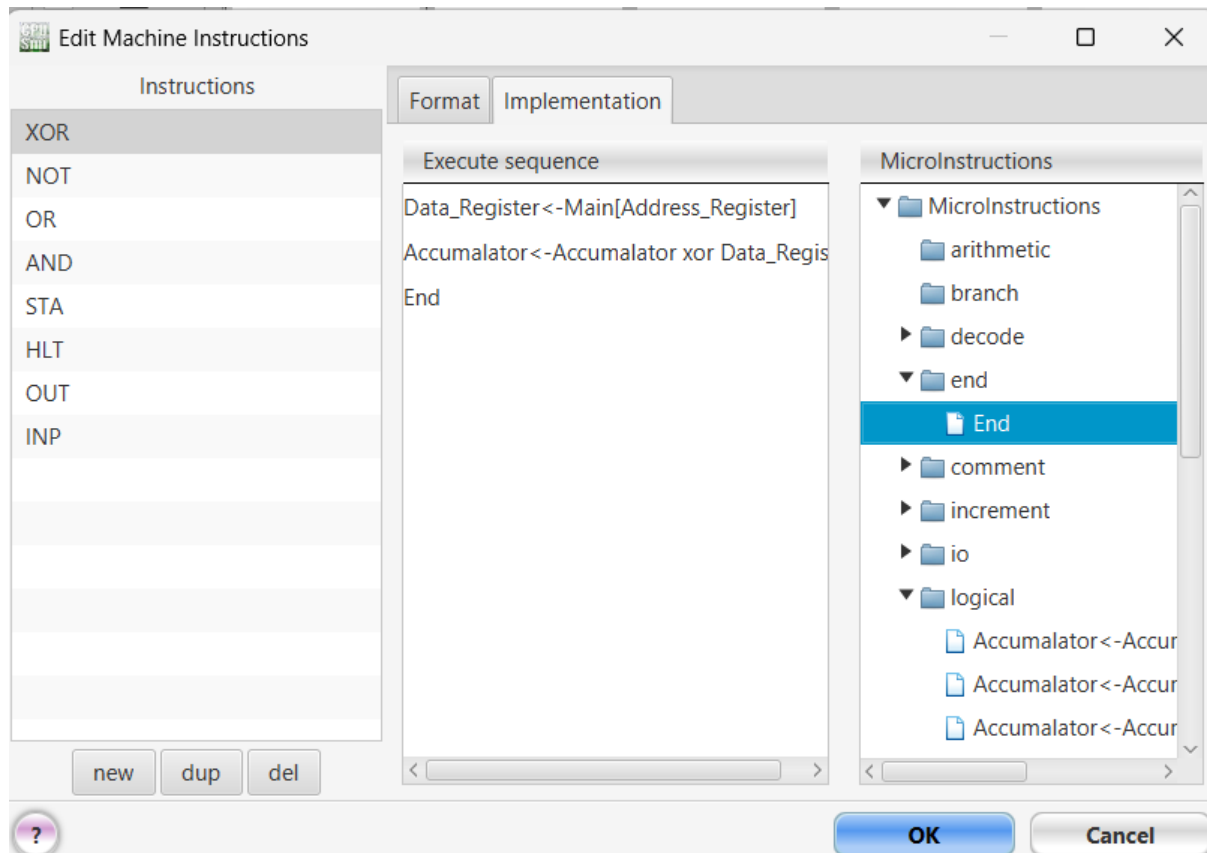
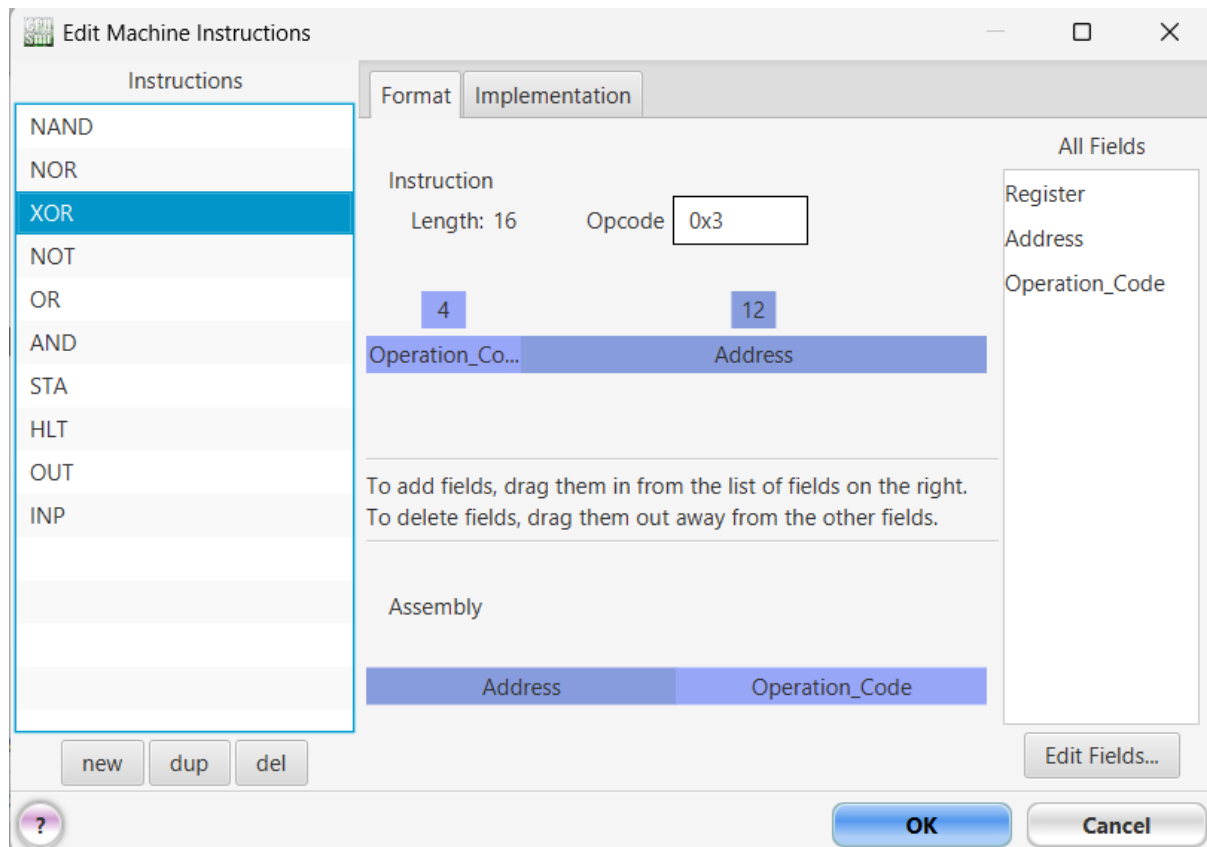




```
EXECUTING...
Enter Inputs, the first of which must be an Integer: 1
Output: -2
EXECUTION HALTED NORMALLY due to the setting of the bit(s): [Halt_Bit]
```

XOR Operation-

Edit Microinstructions				
Type of Microinstruction:		Logical		
name	type	source1	source2	destination
Accumalator<-Ac...	XOR	Accumalator	Data_Register	Accumalator
Accumalator<-Ac...	NOT	Accumalator	Accumalator	Accumalator
Accumalator<-Ac...	AND	Accumalator	Data_Register	Accumalator
Accumalator<-Ac...	OR	Accumalator	Data_Register	Accumalator



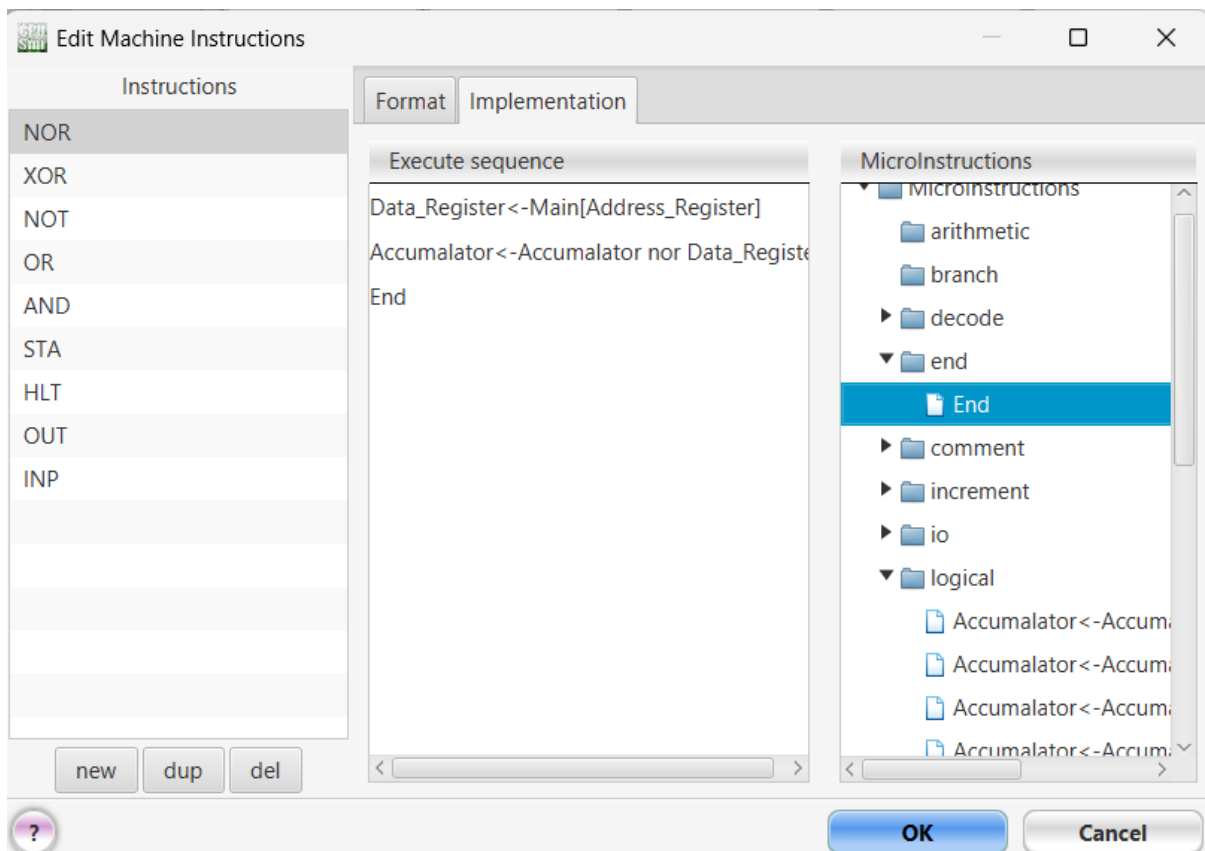
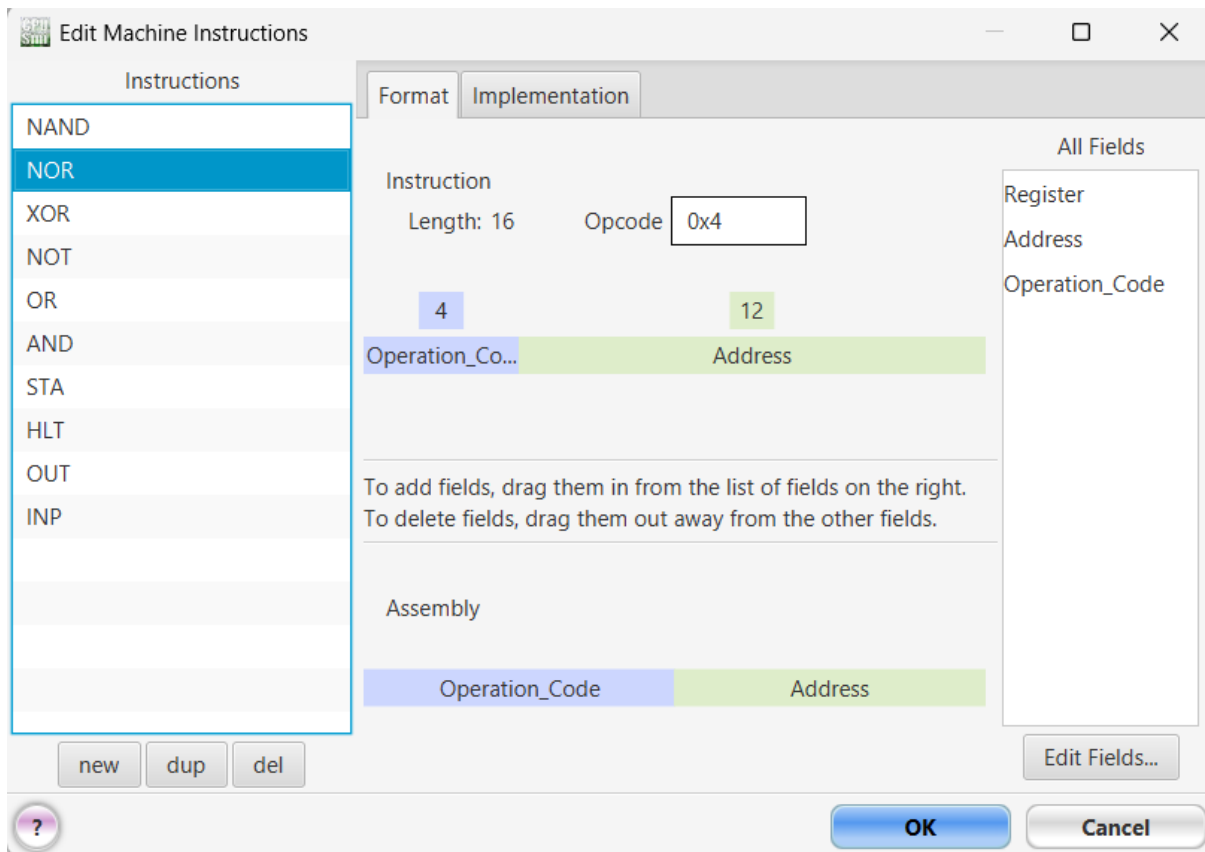
```

xor.a ×
1 INP
2 STA NUM
3 INP
4 XOR NUM
5 OUT
6 HLT
7
8 NUM: .data 1 0
9

```

NOR Operation-

Edit Microinstructions				
Type of Microinstruction: Logical				
name	type	source1	source2	destination
Accumalator<-Ac...	NOR	Accumalator	Data_Register	Accumalator
Accumalator<-Ac...	XOR	Accumalator	Data_Register	Accumalator
Accumalator<-Ac...	NOT	Accumalator	Accumalator	Accumalator
Accumalator<-Ac...	AND	Accumalator	Data_Register	Accumalator
Accumalator<-Ac...	OR	Accumalator	Data_Register	Accumalator



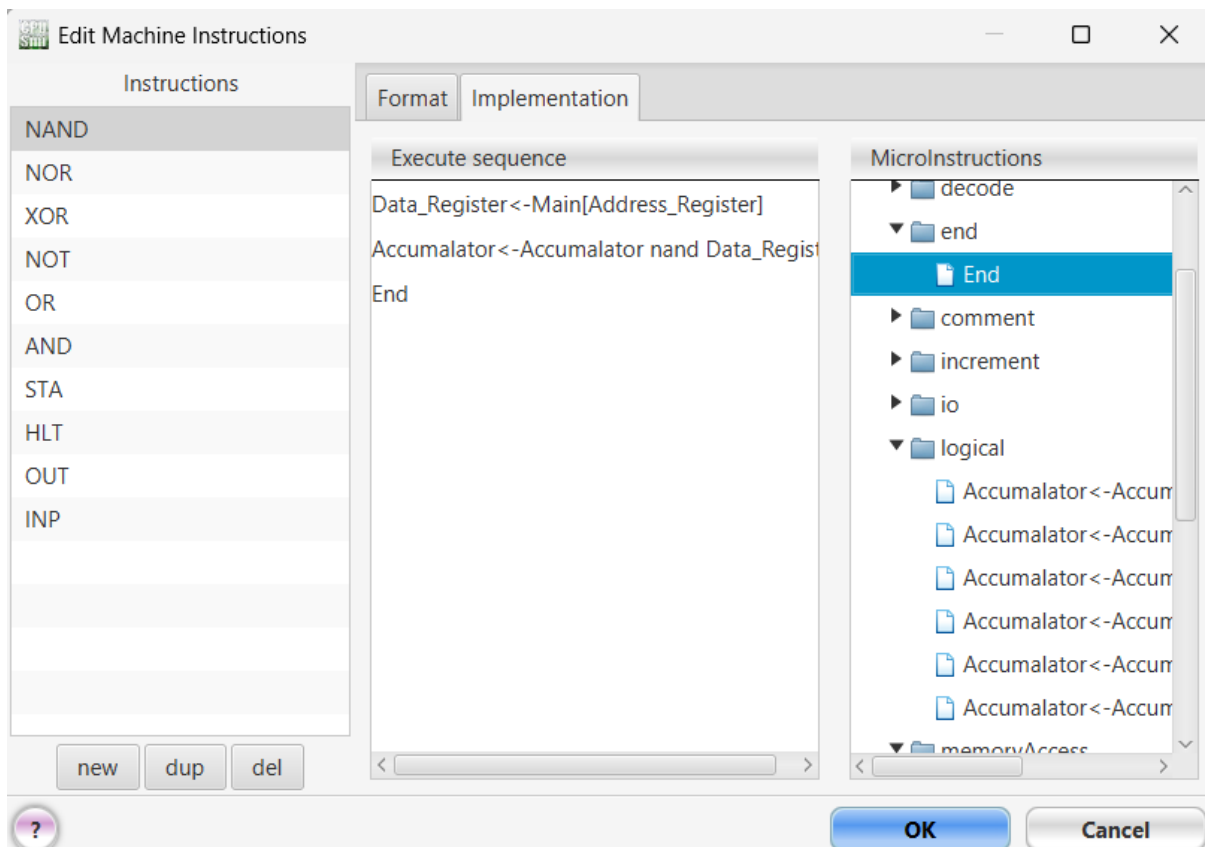
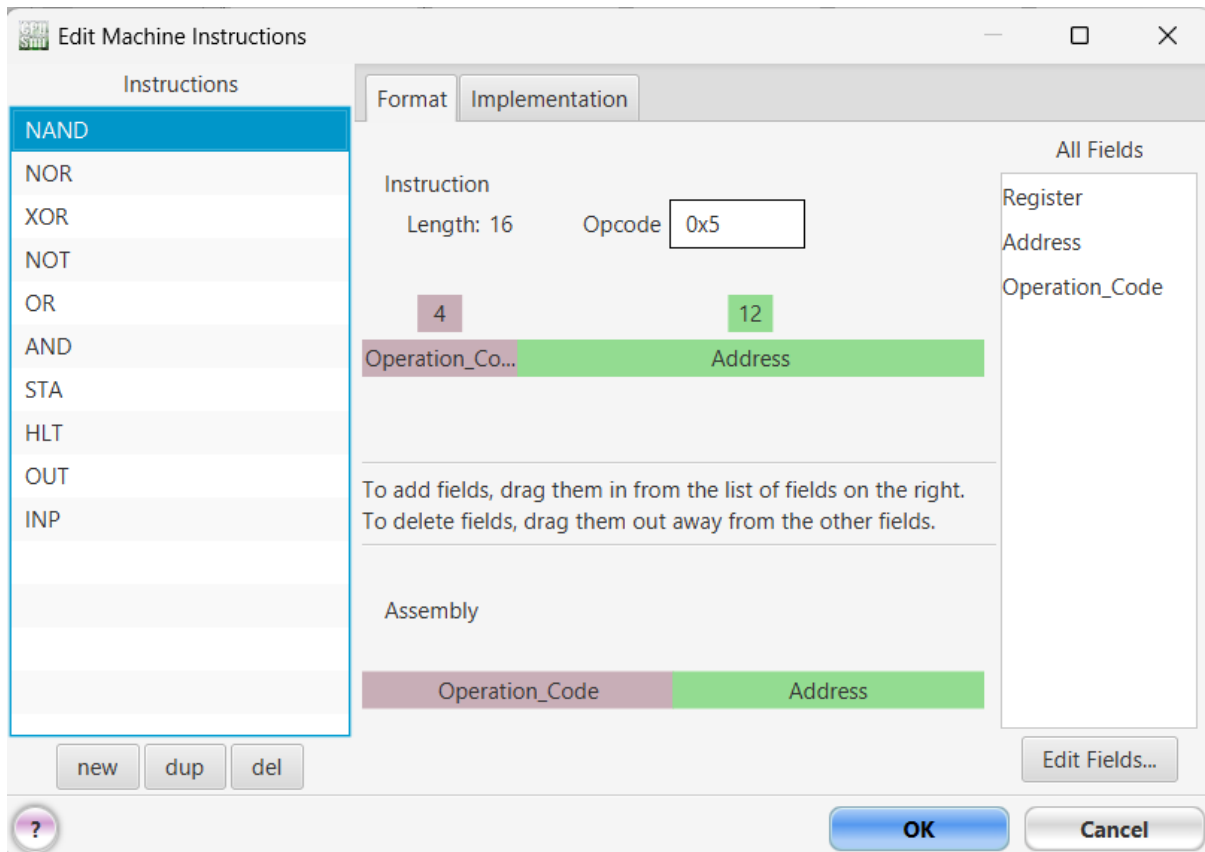
```

nor.a ×
1 INP
2 STA NUM
3 INP
4 NOR NUM
5 OUT
6 HLT
7
8 NUM: .data 1 0
9

```

NAND Operation-

Edit Microinstructions				
Type of Microinstruction: Logical				
name	type	source1	source2	destination
Accumalator<-Ac...	NAND	Accumalator	Data_Register	Accumalator
Accumalator<-Ac...	NOR	Accumalator	Data_Register	Accumalator
Accumalator<-Ac...	XOR	Accumalator	Data_Register	Accumalator
Accumalator<-Ac...	NOT	Accumalator	Accumalator	Accumalator
Accumalator<-Ac...	AND	Accumalator	Data_Register	Accumalator
Accumalator<-Ac...	OR	Accumalator	Data_Register	Accumalator



```
nand.a x
1 INP
2 STA NUM
3 INP
4 NAND NUM
5 OUT
6 HLT
7
8 NUM: .data 1 0
9
```

Results:

All six logical operations were successfully implemented on the 16-bit base machine in CPUSim. The ALU correctly performed each bitwise operation and the results were stored in the Accumulator register.

Learning Outcomes:

After completing this experiment, students are able to:

1. Understand the concept and working of basic logical operations at the bit level.
2. Implement AND, OR, NOT, XOR, NOR, and NAND instructions in CPUSim using microinstruction sequences.
3. Differentiate between unary (NOT) and binary (AND, OR, XOR, NOR, NAND) logical operations.
4. Observe how the ALU processes bitwise operations on 16-bit data stored in registers.
5. Verify the correctness of logical operations using truth tables and simulation output.
6. Understand the importance of logical operations in real-world applications such as masking, encryption, and condition checking.